Repository: Federal Funds Matchmaker (UNC Solvers)

Repository (public): https://github.com/pavparola/Solveathon-Project (branch `main`)

Live prototype: https://app-ffm-web.salmonmushroom-cf3974d2.eastus2.azurecontainerapps.io

UNC Solvers (Team 14): Pavan Parola, Advik Jain, Mayank Puri, Aarav Kulkarni · UNC Solve-a-thon 2026

This file is the repository's README, as a state developer would read it first. The full code, tests (97), deployment scripts and documents are at the link above.

Where things are in the repository

Folder	What is in it
`src/`	The daily job (Grants.gov pull, cleaning, scoring, AI review), the web app, storage and tests helpers
`agency/profiles/`	One profile per agency (NC DHHS, NC DEQ), built from its strategic plan with page citations
`prompts/`	The AI review prompt
`deploy/`	Azure scripts: create resources, build and deploy, tear down
`tests/`	Automated tests
`docs/`	Solution Design, GenAI Field Notes, data contract, tuning report, architecture and threat-model diagrams
`data/`	Qualifier-round results kept as the reference set for tuning

A daily, central tool that helps North Carolina state agencies catch federal funding. Every morning it pulls every posted and forecasted federal opportunity from Grants.gov **once**, ranks it separately against **each agency's own strategic plan**, explains why it matches (with quotes checked against the official listing), and lets each agency's grants coordinator record **Pursue / Maybe / Pass** in seconds. A statewide view shows every agency's matches side by side. The system ranks and explains; people decide.

Agencies today, each from its current plan on the OSBM website: **NC DHHS** (Strategic Plan 2026–2030) and **NC DEQ** (Strategic Plan 2025–2029). Adding an agency is one profile file in the same template.

UNC Solve-a-thon 2026, UNC Solvers (Team 14): Pavan Parola, Advik Jain, Mayank Puri, Aarav Kulkarni. Live prototype: **https://app-ffm-web.salmonmushroom-cf3974d2.eastus2.azurecontainerapps.io** (prototype; not an official NC DHHS website).

Read this	For
`docs/finals_checklist.md`	What the challenge asked for and where each piece is
`docs/solution_design.md`	Architecture, services, data contract, scoring, threats and mitigations
`docs/finals_field_notes.md`	GenAI Field Notes: tools, key prompts, where the AI was wrong
`docs/diagrams/`	Architecture (C4), security (STRIDE) and stakeholder diagrams
`docs/data_contract.md`	Every file in storage and its fields
`docs/qualifier_design.md`	The qualifier pipeline (Layers 0–5) this builds on

How it works

```
Grants.gov API ──▶ daily job (Container Apps Job, 6:00 AM ET) ──▶ Blob Storage (JSON) ──▶ web app (Container App) ──▶ staff
USAspending.gov ──▶   ingest → normalize → score → AI rationales → alerts → run log       reads results, saves decisions
                                  ▲ Azure OpenAI: embeddings (all) + gpt-4.1-mini (top 45)
```

1. **Ingest** (`src/grantsgov.py`): every posted and forecasted listing (~1,460) and its full detail; raw responses kept by date.
2. **Normalize** (`src/normalize.py`): one schema; the qualifier's tested rules for dates, placeholder amounts, cost share, eligibility and research grants; closed and duplicate records dropped; incomplete ones flagged.
3. **Score** (`src/matcher.py`), once per agency profile: 0–100 = 45% plan alignment + 25% keyword fit + 15% eligibility (state agency) + 10% time to apply + 5% funder history (USAspending.gov). High ≥ 55, Medium ≥ 45. Research grants and records that exclude state agencies go to separate lists.

4. **Explain** (`src/rationale.py`, `prompts/rationale.txt`): the AI writes a summary, why it matches, eligibility, deadline and the biggest risk; code checks every quote word for word. The AI never sets the score.
5. **Serve** (`src/webapp.py`): an agency switcher; per agency a ranked shortlist with filters, detail page, decisions, CSV export, funding by goal, evaluation (blind labeling, precision@10 vs a keyword baseline); a statewide view across agencies; run status.

Run it on a laptop

Python 3.12+.

```
pip install -r requirements.txt
python src/job.py          # pulls live data into data/store/ (about 1 minute; AI steps skipped without Azure OpenAI)
python src/webapp.py       # http://localhost:8000
python -m pytest tests -q  # 97 tests
```

Without Azure OpenAI the job uses TF-IDF matching and shows the score breakdown instead of AI summaries. To use Azure OpenAI locally, set the variables below in your shell or in a `.env` file (git-ignored). `python src/job.py --replay 2026-11-07` re-runs on a saved day without calling Grants.gov.

Environment variables

Variable	Used by	Meaning
`MODE`	container	`app` (web app, default) or `job` (run the daily pipeline once and exit)
`STORAGE_CONN`	both	Azure Storage connection string (**secret**). Unset → local folder `data/store/`
`STORAGE_CONTAINER`	both	Blob container, default `ffm-data`
`AOAI_ENDPOINT`	job	Azure OpenAI endpoint
`AOAI_KEY`	job	Azure OpenAI key (**secret**)
`AOAI_EMBED_DEPLOYMENT`, `AOAI_CHAT_DEPLOYMENT`	job	Deployment names, default `embed` and `chat`
`FLASK_SECRET_KEY`	app	Session key so every worker accepts the same form tokens (**secret**)

Secrets live only as Container Apps secrets in Azure. Never put them in the repository.

Deploy to Azure

Follows the starter kit's steps with the Azure CLI (works in Azure Cloud Shell or any terminal after `az login`). Names are in `deploy/env.sh`.

```
bash deploy/01_resources.sh          # resource group, Blob Storage, Azure OpenAI + 2 models, registry, Container Apps environment
bash deploy/02_deploy.sh             # build the image in the cloud, deploy the web app and the daily job, run the job once
TAG=v9 bash deploy/02_deploy.sh      # roll out a code change
bash deploy/99_teardown.sh           # after judging: delete everything (asks for confirmation)
```

Run the job now and watch it:

```
az containerapp job start -n job-ffm-refresh -g rg-ffm-team14
az containerapp job execution list -n job-ffm-refresh -g rg-ffm-team14 -o table
```

Add another agency

1. Copy `agency/profiles/ncdeq.json` to `agency/profiles/<id>.json` and fill it in from that agency's strategic plan on the OSBM website: mission, goals and objectives (each with its plan page), divisions, and the `profile` block (`agency_id`, plan title and link, program areas, 20–40 keywords, eligible applicant types, the funders that usually support it, the name pattern USAspending uses for it, and ten "unrelated fields" for the matcher).
2. Have someone who did not write it check it against the plan.
3. Run the tests and redeploy. The next daily run scores every opportunity for the new agency too; it appears in the agency switcher and the statewide view. Nothing else changes.

When an agency publishes a new plan, update its profile the same way. NC DHHS published its 2026–2030 plan in September 2026; we rebuilt its profile from it during the event (the qualifier round used the 2023–2025 plan, which `agency/priorities.json` keeps for reproducing the qualifier results).

Known limitations

- Each profile is only as current as the agency's plan; when a plan changes, the profile must be rebuilt (done for NC DHHS 2026–2030).
- Grants.gov only: no SAM.gov, foundations or state pass-through funds yet.
- Short or vague listings can score low (false negatives); Low matches are hidden by default but one click away.
- No sign-in in the prototype: anyone with the URL can record a decision. A pilot would add Entra ID sign-in (Container Apps built-in authentication), Key Vault with managed identity, and Application Insights.
- The matcher was tuned on NC DHHS reference data; NC DEQ uses the same settings untuned.
- Statutory authority is not modelled yet: the profile records divisions and their scope, not the statutes behind them.

Repository map

`src/`	`job.py (daily pipeline) · webapp.py + templates/ + static/ (staff app) · grantsgov.py · normalize.py matcher.py · rationale.py · evaluate.py · storage.py · aoai.py · qualifier layers (clean_data.py, deterministic.py, award_history.py, …)`
`agency/`	`profiles/*.json: one reviewed profile per agency (NC DHHS, NC DEQ); priorities.json: the qualifier's profile`
`prompts/`	`rationale.txt (finals) · qualifier prompts`
`deploy/`	`Azure CLI scripts and the container entrypoint`
`docs/`	`design, field notes, data contract, checklist, diagrams, qualifier design`
`tests/`	`96 tests (python -m pytest tests -q)`
`submission/`	`qualifier submission files`